Manipal School of Information Sciences
AME 5151 - Applied Linear Algebra Lab

Assignment 2

Academic Year: 2026-2027, Semester: 1

Tuesday, 15 September, 2026

This assignment maps to the lab course outcome **CO2**:

Implement models for real-life applications using linear algebra principles and analyze the results from a practical perspective.

Total Points: **50**

Semantic Tagging with GloVe and NumPy

You have already built a Vector class using tuples for storage and have implemented operations such as add, scale, dot, norm, and cosine_similarity.

In this lab, you will apply the same vector operations to word embeddings. We will use the pretrained GloVe model glove-wiki-gigaword-50. Each word represented by this model is associated with a vector in $\mathbb{R}^{50}$.

The main objective of this lab is to move from computing cosine similarity between one pair of vectors to computing cosine similarities between two collections of vectors.

Suppose that, after preprocessing, the input text contains n words that are present in the model vocabulary. Arrange their embedding vectors as the rows of a matrix $W \in \mathbb{R}^{n\times 50}$.

Similarly, arrange the vectors corresponding to 20 predefined tags as the rows of a matrix $T \in \mathbb{R}^{20\times 50}$.

After normalizing every row of W and T to have Euclidean norm one, define

$$\widehat{W} \in \mathbb{R}^{n\times 50}, \qquad \widehat{T} \in \mathbb{R}^{20\times 50}.$$

The matrix $S = \widehat{W}\widehat{T}^{\,T}$ has dimensions $S \in \mathbb{R}^{n\times 20}$.

Its entries satisfy

$$S_{ij} = \widehat{W}_{i,:}\, \widehat{T}_{j,:}^{T} = \cos(W_{i,:}, T_{j,:}).$$

Thus, S_{ij} is the cosine similarity between the i-th word in the processed text and the j-th tag.

This is the central linear-algebra operation in the lab.

Learning Objectives

By the end of this lab, you should be able to:

1. Represent a fixed vocabulary of 20 tags as a matrix

$$T \in \mathbb{R}^{20\times 50}.$$

2. Preprocess an input text and represent its in-vocabulary words as a matrix

$$W \in \mathbb{R}^{n\times 50}.$$

3. Compute cosine similarity between every in-vocabulary text word and every tag.

4. Construct the complete similarity matrix

$$S \in \mathbb{R}^{n\times 20}.$$

5. Compute the same similarity matrix in two ways:

   (a) using your own Vector class and Python loops;

   (b) using NumPy matrix operations.

6. Verify numerically that the two implementations compute the same cosine-similarity values, up to normal floating-point error.

7. Use the similarity matrix to rank the 20 tags according to their relevance to the input text.

Given Input and Required Components

Use the pretrained embedding model glove-wiki-gigaword-50. The model associates every word in its vocabulary with a 50-dimensional NumPy vector.

You must implement:

1. a vocabulary of exactly 20 tags.
2. a preprocessing pipeline for the input text.
3. construction of the tag matrix T.
4. construction of the text matrix W.
5. construction of the similarity matrix using your Vector class.
6. construction of the similarity matrix using NumPy.
7. numerical verification of the two results.
8. tag ranking using max pooling.

Do not use a library function that directly computes the complete cosine-similarity matrix.

Part A — Choose 20 Tags

Choose exactly 20 tags that you consider appropriate for describing activities associated with a research university.

Each tag must represent an academic, research, educational, professional, institutional, or campus-related concept.

A possible starting vocabulary is: *research*, *innovation*, *education*, *university*, *students*, *faculty*, *campus*, *engineering*, *medicine*, *technology*, *curriculum*, *collaboration*, *publication*, *laboratory*, *scholarship*, *mentorship*, *internship*, *entrepreneurship*, *accreditation*, *alumni*.

You may use this list or construct your own list.

In your documentation, briefly justify your choice of tag vocabulary.

Single-word tags

If a tag consists of a single word t, its tag vector is

$$\mathbf{v}_t = \texttt{model[t]}.$$

Every single-word tag must be present in the model vocabulary.
For example:

```
    if tag not in model.key_to_index:
        raise ValueError(f"Tag not in vocabulary: {tag}")
```

Multi-word tags

You may use a multi-word tag such as

$$\texttt{higher education}.$$

Suppose a tag consists of k words

$$t_1, t_2, \ldots, t_k.$$

Represent the tag by the arithmetic mean of the individual word vectors:

$$\mathbf{v}_{\text{tag}} = \frac{1}{k} \sum_{r=1}^{k} \mathbf{v}_{t_r}.$$

Every component word of a multi-word tag must be present in the model vocabulary. If any component word is absent, choose a different tag.
Document all multi-word tags used in your vocabulary.

Part B — Build the Input Text

Collect at most 400 words of real text concerning Manipal University from LinkedIn posts.
Cite the source of every excerpt used.
Save the collected material in a plain-text file with extension `.txt`.
The text may contain:

- punctuation;
- names;

- numbers;
- hashtags;
- repeated words;
- words that do not occur in the embedding vocabulary.

Your program must handle such input without assuming that every token has a corresponding embedding vector.

Part C — Preprocess the Text

Apply the following preprocessing steps in the stated order:

1. convert the text to lowercase;
2. remove punctuation;
3. split the resulting text into tokens;
4. remove stopwords using a small stopword set that you define;
5. remove tokens whose length is less than or equal to 2;
6. determine which remaining tokens occur in the embedding vocabulary.

Do not use an external stopword library. Define a small stopword set yourself. For example:

```
STOPWORDS = {
    "the", "a", "an", "and", "or", "of", "to",
    "in", "on", "for", "is", "are", "was", "were",
    "with", "at", "by", "from"
}
```

This starter set is deliberately incomplete. Real LinkedIn text will also contain pronouns, demonstratives, and other function words (`it`, `this`, `that`, `our`, `we`, and similar). Expand the set as needed for your own text and note in your documentation what you added.

Let

$$x_1, x_2, \ldots, x_m$$

denote the tokens remaining after the first five preprocessing steps.
Some of these tokens may not be present in the model vocabulary.
Define the in-vocabulary token sequence as

$$w_1, w_2, \ldots, w_n, \qquad n \leq m,$$

where each w_i is present in the model vocabulary.
Your program must report:

- the number of tokens before preprocessing;
- the number of tokens after preprocessing;
- the number of in-vocabulary tokens;
- the number of out-of-vocabulary tokens;
- the distinct out-of-vocabulary tokens.

Repeated in-vocabulary words must *not* be removed. If a word appears several times in the processed text, its vector must appear the same number of times in the text matrix.

Part D — Build the Tag and Text Matrices

Implement the following functions.

```
def build_tag_matrix(model, tags):
    """
    Construct the tag matrix.

    Returns:
        tag_names:
            list[str] containing exactly 20 tag names.

        T:
            NumPy array with shape (20, 50).

    For a single-word tag:
        the corresponding row of T is model[tag].

    For a multi-word tag:
        the corresponding row is the arithmetic mean
        of the vectors of all component words.

    Every word used in a tag must occur in the model
    vocabulary. Otherwise, raise an error.
    """

    def build_text_matrix(model, tokens):
    """
    Construct the text matrix.

    Parameters:
        tokens:
            preprocessed tokens obtained after lowercasing,
            punctuation removal, stopword removal, and the
            length filter.

    Returns:
        in_vocab_tokens:
            tokens that occur in the model vocabulary,
            preserving their original order and repetitions.

        oov_tokens:
            tokens that do not occur in the model vocabulary.
```

```
40
41              W:
42                  NumPy array with shape (n, 50), where n is the
43                  number of in-vocabulary tokens.
44
45          Row i of W is the embedding vector of
46          in_vocab_tokens[i].
47          """
```

If

$$w_1, w_2, \ldots, w_n$$

are the in-vocabulary tokens, then

$$W = \begin{bmatrix} - & \mathbf{w}_1^T & - \\ - & \mathbf{w}_2^T & - \\ & \vdots & \\ - & \mathbf{w}_n^T & - \end{bmatrix} \in \mathbb{R}^{n \times 50}.$$

Similarly,

$$T = \begin{bmatrix} - & \mathbf{t}_1^T & - \\ - & \mathbf{t}_2^T & - \\ & \vdots & \\ - & \mathbf{t}_{20}^T & - \end{bmatrix} \in \mathbb{R}^{20 \times 50}.$$

Checkpoint

Print:

```
1   print(T.shape)
2   print(W.shape)
```

Verify that

$$T.\text{shape} = (20, 50)$$

and

$$W.\text{shape} = (n, 50).$$

If $n = 0$, the input contains no usable words and the program must terminate with an explanatory error message rather than attempting the similarity computation.

Part E — Cosine Similarity Using Your Vector Class

Recall that for nonzero vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{50}$,

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}.$$

First compute the complete similarity matrix using your own `Vector` class. Implement:

```python
def similarity_matrix_vector_class(W, T):
    """
    Compute the complete cosine-similarity matrix using
    the previously implemented Vector class.

    Every entry MUST be computed by constructing two
    Vector instances and calling their cosine_similarity
    method -- not by calling NumPy's dot/norm functions
    directly on the rows of W and T.

    Returns:
        S with shape (n_words, 20), where

        S[i, j] = Vector(W[i]).cosine_similarity(Vector(T[j])).

    This implementation may use Python loops.
    NumPy matrix multiplication must not be used to
    compute the similarities in this function.
    """
```

For every i and j,

$$S_{ij} = \frac{W_{i,:}\, T_{j,:}^{T}}{\|W_{i,:}\|_2 \|T_{j,:}\|_2}.$$

The resulting matrix must have dimensions

$$S_{\text{Vector}} \in \mathbb{R}^{n \times 20}.$$

This implementation serves as the reference computation using the vector operations that you implemented earlier in the course.

Part F — Cosine Similarity Using NumPy

Now compute the same matrix without using a Python-level double loop over all text-word/tag pairs.

First normalize every row of W:

$$\widehat{W}_{i,:} = \frac{W_{i,:}}{\|W_{i,:}\|_2}.$$

Similarly, normalize every row of T:

$$\widehat{T}_{j,:} = \frac{T_{j,:}}{\|T_{j,:}\|_2}.$$

Then compute

$$\boxed{S_{\text{NumPy}} = \widehat{W}\widehat{T}^{\,T}}$$

using NumPy matrix multiplication.

The matrix dimensions are

$$\underbrace{\widehat{W}}_{n\times 50}\ \underbrace{\widehat{T}^{\,T}}_{50\times 20} = \underbrace{S_{\text{NumPy}}}_{n\times 20}.$$

Implement:

```
def similarity_matrix_numpy(W, T):
    """
    Compute all text-word/tag cosine similarities using
    NumPy row normalization and matrix multiplication.

    Returns:
        S with shape (n_words, 20).
    """
```

A suitable implementation has the following mathematical structure:

```
W_norm = np.linalg.norm(W, axis=1, keepdims=True)
T_norm = np.linalg.norm(T, axis=1, keepdims=True)

W_hat = W / W_norm
T_hat = T / T_norm

S = W_hat @ T_hat.T
```

Before division, verify that the row norms are nonzero.

Do not call a library cosine-similarity function for this computation.

Part G — Verify the Two Implementations

The two similarity matrices must represent the same mathematical quantities.

Before comparing them, convert S_{Vector} to a NumPy array (e.g. `S_vector = np.array(S_vector)`) if your `Vector`-class implementation returned a nested list.

Compare them using

```
np.allclose(S_vector, S_numpy)
```

with an appropriate floating-point tolerance.

The result should be `True`.

Because floating-point arithmetic is approximate, do not require the corresponding entries to be bit-for-bit identical.

Also report

```
np.max(np.abs(S_vector - S_numpy))
```

to show the largest absolute numerical difference between the two implementations.

Part H — Rank Tags Using Max Pooling

The j-th column of S contains the similarities between tag j and every in-vocabulary word in the text:

$$S_{1j}, S_{2j}, \dots, S_{nj}.$$

Define the max-pool score for tag j as

$$r_j = \max_{1 \le i \le n} S_{ij}.$$

Thus,

$$\boxed{r_j = \max_i S_{ij}}$$

is the similarity between tag j and the single text word having the highest cosine similarity to that tag.

Implement:

```
    def rank_tags(tag_names, S, in_vocab_tokens):
        """
        For each tag j:

            score[j] = max_i S[i, j]

        Return the tags sorted in descending order of score.

        For each tag, also return the text token that produced
        the maximum score.
        """
```

For each of the top eight tags, display:

- the rank;
- the tag;
- its max-pool similarity score;
- the text word that produced that score.

For example, the output format may be:

```
Rank   Tag              Score      Best-matching text word
-------------------------------------------------------------
1      research         0.91       research
2      innovation       0.84       innovative
3      technology       0.79       digital
...
```

Your numerical values will depend on the text and the selected tag vocabulary.

Interpretation of Max Pooling

Max pooling answers the following precise question:

> For this tag, which single word in the document has the highest cosine similarity to the tag?

A high max-pool score does *not* imply that the corresponding topic dominates the document. A tag can receive a high score because of a single strongly related word.

If a text word is identical to a tag (e.g. the word "research" appears verbatim and "research" is one of your tags), that pair gets a similarity of exactly 1, since cosine similarity of a vector with itself is 1. On real text this will happen often, and such exact matches will tend to dominate the top of your ranking ahead of tags that are only captured through a genuinely different, semantically related word. Note in your output which top-ranked tags were matched by an identical word versus a different one.

Therefore, max-pool ranking measures the presence of strong local semantic evidence for a tag; it does not measure how frequently or how extensively the corresponding topic occurs throughout the text.

Required Unit Tests

Write unit tests for at least the following cases.

1. Verify that
$$T.\text{shape} = (20, 50).$$
2. Verify that if the text contains n in-vocabulary tokens, then
$$W.\text{shape} = (n, 50).$$
3. Verify that
$$S.\text{shape} = (n, 20).$$
4. Select at least one pair consisting of a text word and a tag. Compute its cosine similarity using your `Vector` class and verify that it agrees with the corresponding entry of the NumPy similarity matrix.
5. Verify that
```
        np.allclose(S_vector, S_numpy)

```
returns `True`.
6. Verify that an out-of-vocabulary text token is excluded from W and is reported as out-of-vocabulary.
7. Verify that a tag vocabulary containing an out-of-vocabulary component causes `build_tag_matrix` to raise an error (it must not silently drop the offending tag and proceed with the remaining 19).

Questions for the Lab Report

Answer the following questions precisely.

1. If there are n in-vocabulary words in the processed text, give the dimensions of:
   (a) one word vector;
   (b) W;
   (c) T;
   (d) T^T;
   (e) S.
2. Explain why, after row normalization,
   $$\widehat{W}_{i,:}\widehat{T}_{j,:}^{T}$$
   is equal to the cosine similarity between the corresponding original vectors.
3. Explain why
   $$\widehat{W}\widehat{T}^{T}$$
   computes all $20n$ pairwise text-word/tag cosine similarities in one matrix multiplication.
4. Suppose a single word in the document has very high similarity to the tag `medicine`, while all other words have low similarity to that tag. Explain how max pooling treats this situation.
5. Does a high max-pool score imply that the corresponding tag describes the dominant topic of the document? Explain.
6. For your own top-eight ranking, identify which tags were matched by a text word identical to the tag itself, and which were matched by a genuinely different word. What does the difference tell you about what cosine similarity over word embeddings is, and is not, capturing?
7. If a concept is not represented by any of the 20 chosen tags, can the ranking algorithm introduce a new tag for that concept? Explain.
8. Why are out-of-vocabulary tokens excluded from W?
9. Explain the mathematical relationship between your pair-by-pair `Vector.cosine_similarity` computation and the NumPy matrix multiplication used to construct S.

Submission Requirements

Submit:

1. the source text file containing at most 400 words;
2. citations for all LinkedIn excerpts used;
3. your Python source code;
4. your unit tests;
5. program output showing:
    - token statistics;
    - T.shape;
    - W.shape;
    - verification using `np.allclose`;
    - the maximum numerical difference between the two similarity matrices;
    - the top eight ranked tags and their best-matching text words;
6. answers to the lab-report questions.

Rubric — 50 Points

Component	Points
Tag vocabulary and tag-matrix construction	8
Text preprocessing and OOV handling	7
Similarity matrix using the `Vector` class	8
Vectorized NumPy similarity matrix	10
Max-pool tag ranking	6
Unit tests and numerical verification	7
Documentation and lab-report answers	4
Total	**50**